Threads

Chapter 4

Process – Scheduling Unit and Resource Ownership

- Scheduling/execution- follows an execution path that may be interleaved with other processes
- Resource ownership - process includes a virtual address space to hold the process image
- These two characteristics are treated independently by the operating system

Multithreading

- Modern operating system supports multiple threads of execution within a single process

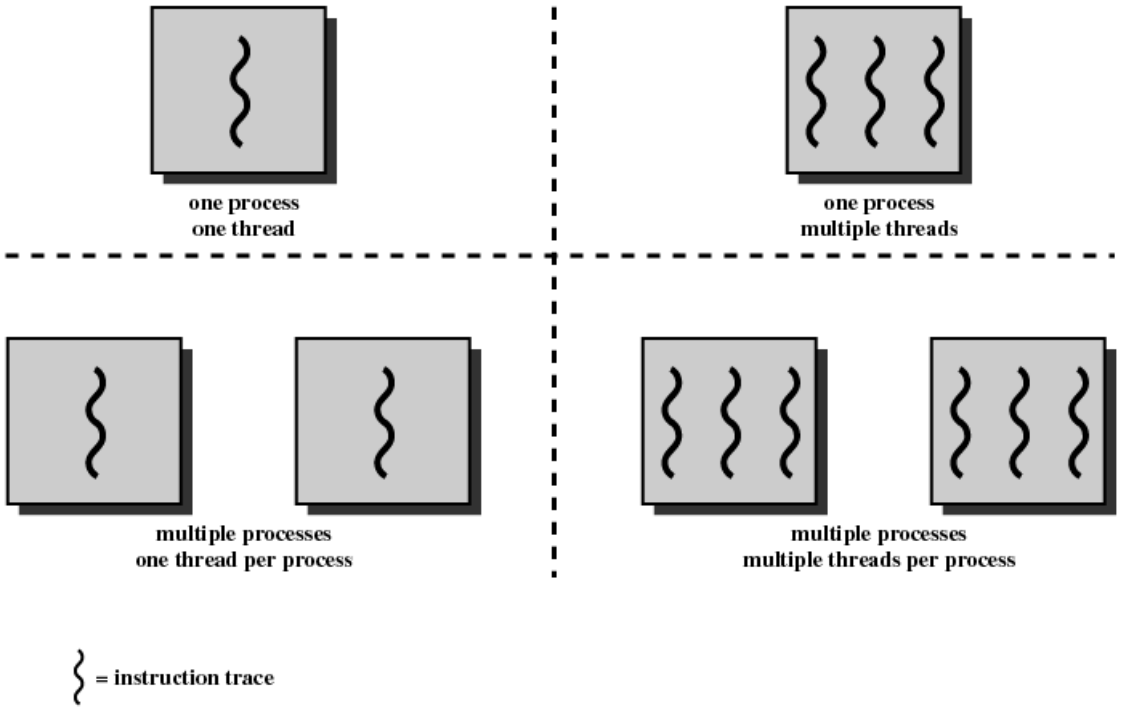


Figure 4.1 Threads and Processes [ANDE97]

Multithreading (contd.)

- In a multithreaded environment, a process is defined as the unit of resource allocation and a unit of protection:
  - Have a virtual address space which holds the process image
  - Protected access to processors, other processes (for interprocess communication), files, and I/O resources (devices and channels)

Multithreading (contd.)

- Within a process, there may be one or more threads, each with the following:
  - An execution state (running, ready, etc.)
  - Saved thread context when not running
  - Has an execution stack
  - Some per-thread static storage for local variables
  - Access to the memory and resources of its process

Single Threaded and Multithreaded Process Models

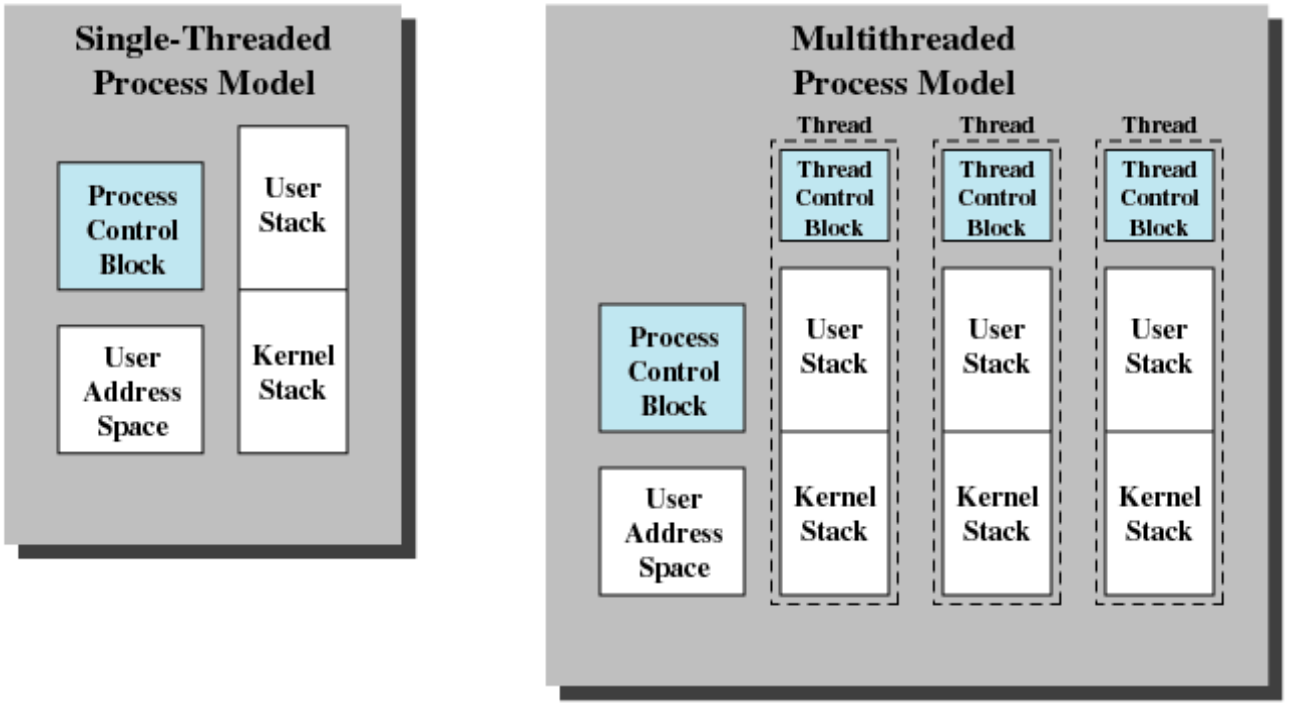
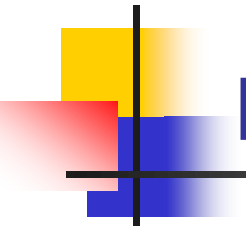
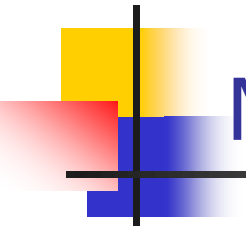


Figure 4.2 Single Threaded and Multithreaded Process Models



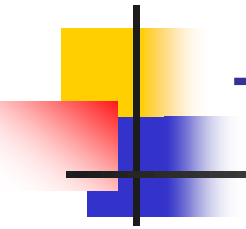
Benefits of Multithreading

- Takes less time to create a new thread than a process
- Less time to terminate a thread than a process
- Less time to switch between two threads within the same process
- Since threads within the same process share memory and files, they can communicate with each other without invoking the kernel



Multithreaded Scenarios

- Foreground to background work
- Asynchronous processing
- Speed of execution
- Modular program structure

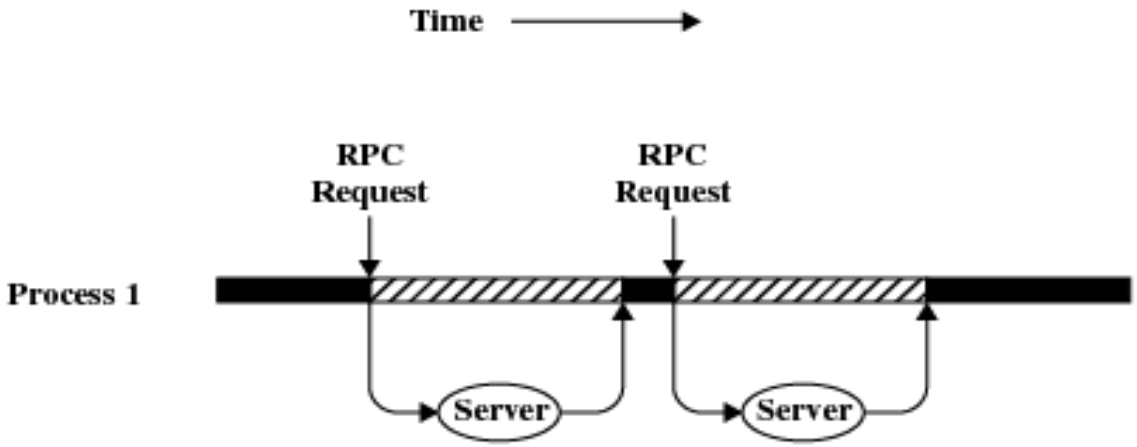


Thread States

- States associated with a change in thread state
  - Spawn
    - Spawn another thread
  - Block
  - Unblock
  - Finish
    - Deallocate register context and stacks

Multithreading Example - RPC

- A program that performs two remote procedure calls (RPCs) to two different hosts to obtain a combined result
  - RPCs use single thread
  - RPCs use multiple threads



(a) RPC Using Single Thread

Multithreading Example - RPC

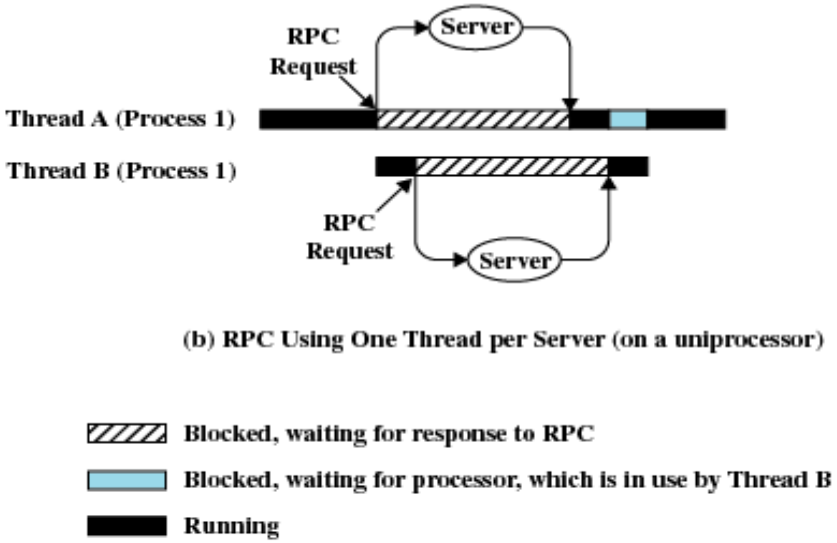


Figure 4.3 Remote Procedure Call (RPC) Using Threads

Multithreading Example

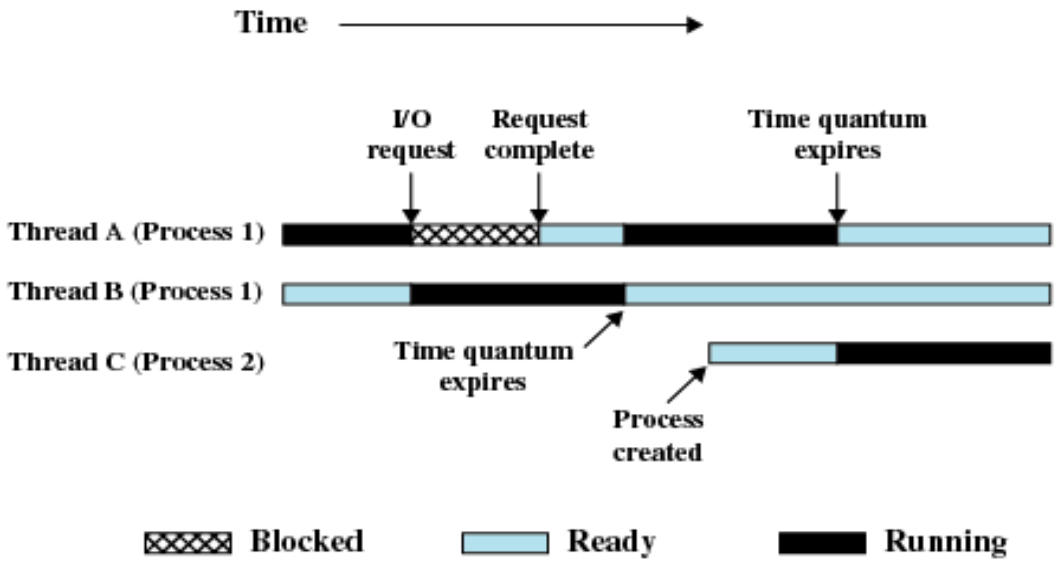


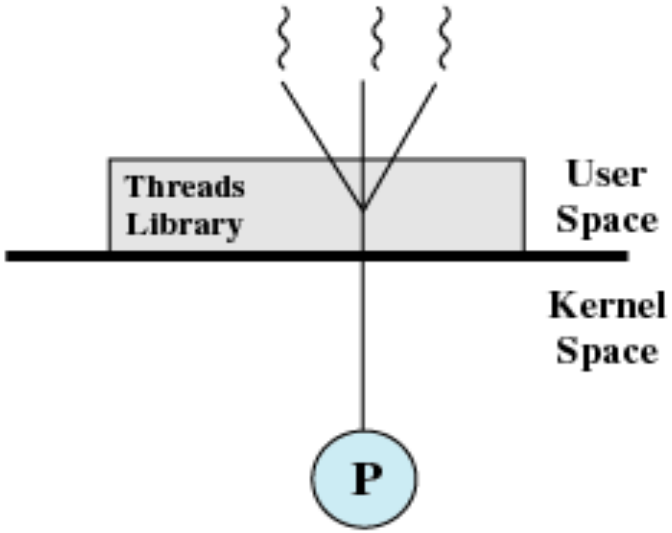
Figure 4.4 Multithreading Example on a Uniprocessor

Different Threading Implementations

- User-level threads (用户级多线程)
- Kernel-level threads (核心级多线程)

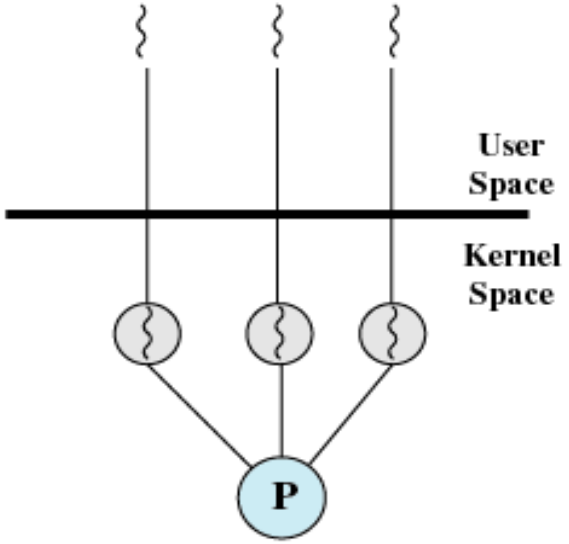
User-Level Threads

- All thread management is done by the application
- The kernel is not aware of the existence of threads



Kernel-Level Threads

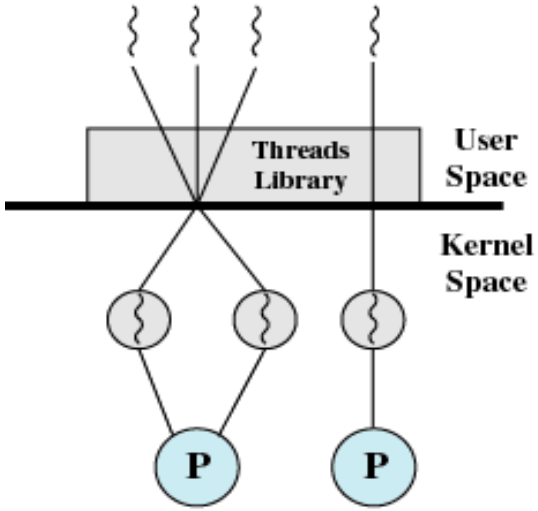
- Kernel maintains context information for the process and the threads
  - Windows is an example of this approach
- Scheduling is done on a thread basis



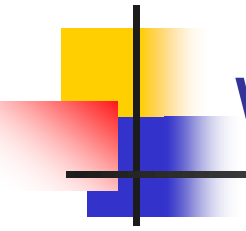
(b) Pure kernel-level

Combined Approaches

- Thread creation done in the user space
- Bulk of scheduling and synchronization of threads within application
  - Example is Solaris



(c) Combined



Windows Thread And SMP

- Windows process are implemented as objects
- An executable process may contain one or more threads
- Both processes and thread objects have built-in synchronization capabilities

Windows Process and its Resources

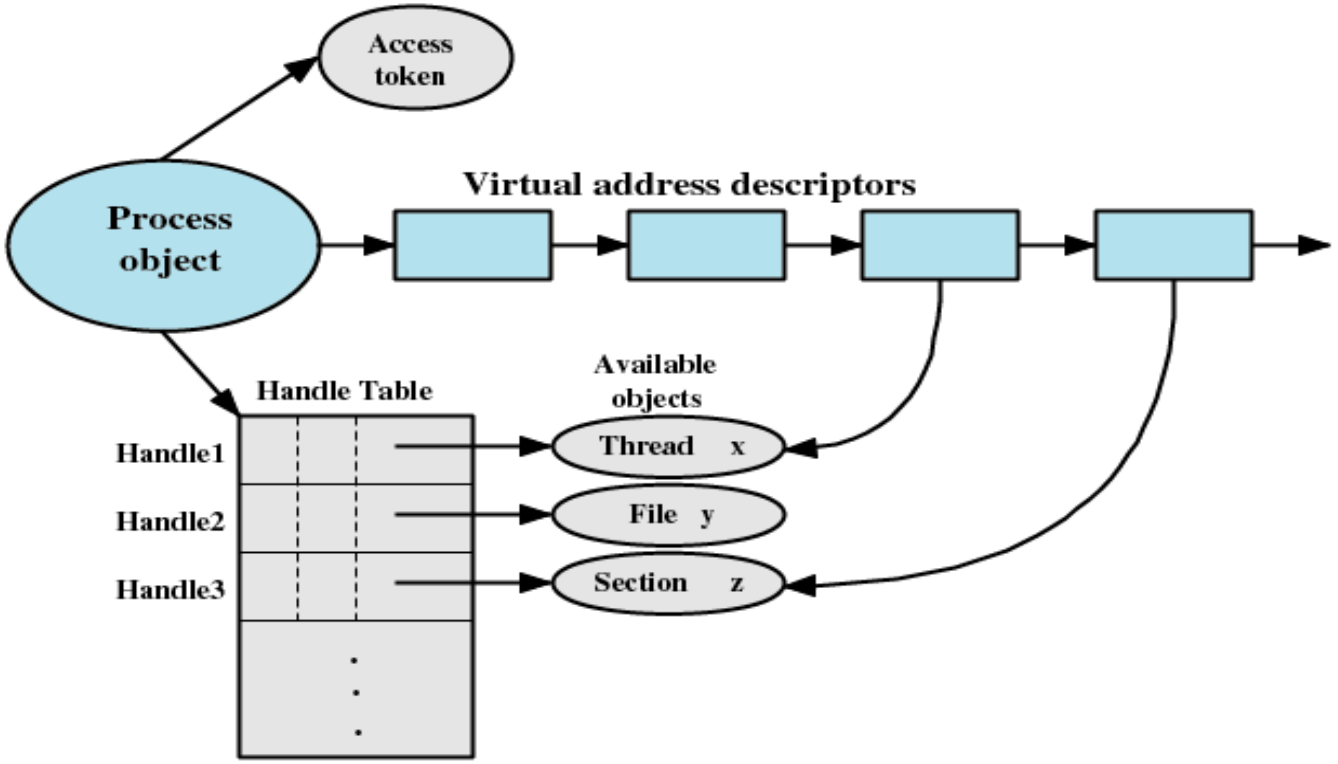
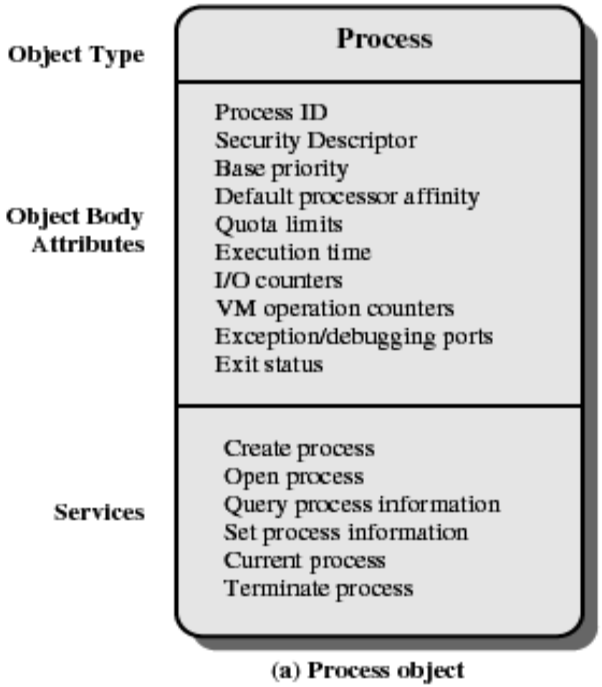


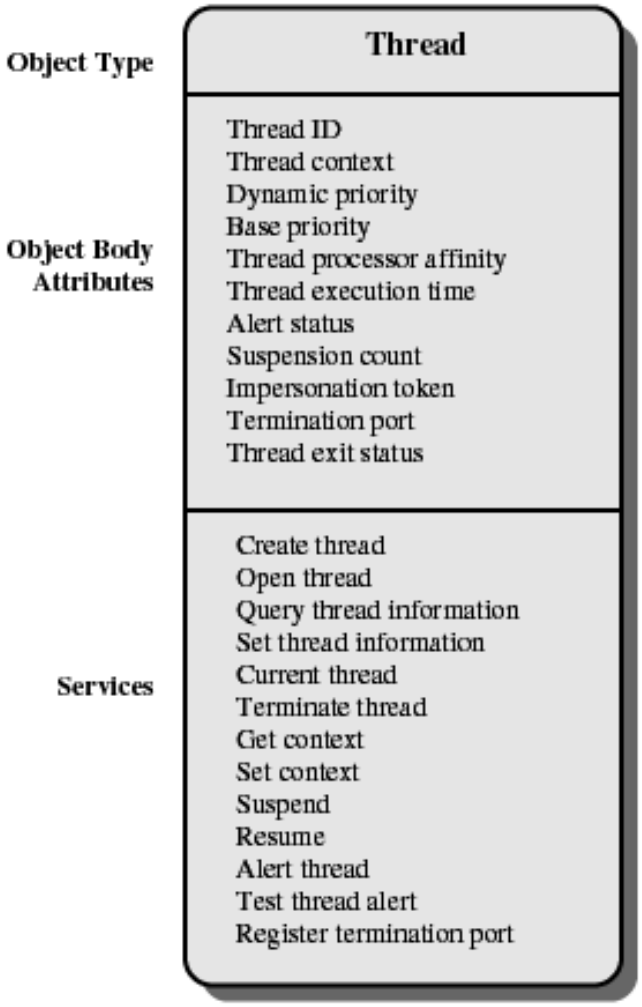
Figure 4.12 A Windows Process and Its Resources

Windows Process Object



(a) Process object

Windows Thread Object



(b) Thread object

Windows 2000 Thread States

- Ready
- Standby
- Running
- Waiting
- Transition
- Terminated

Windows Thread States

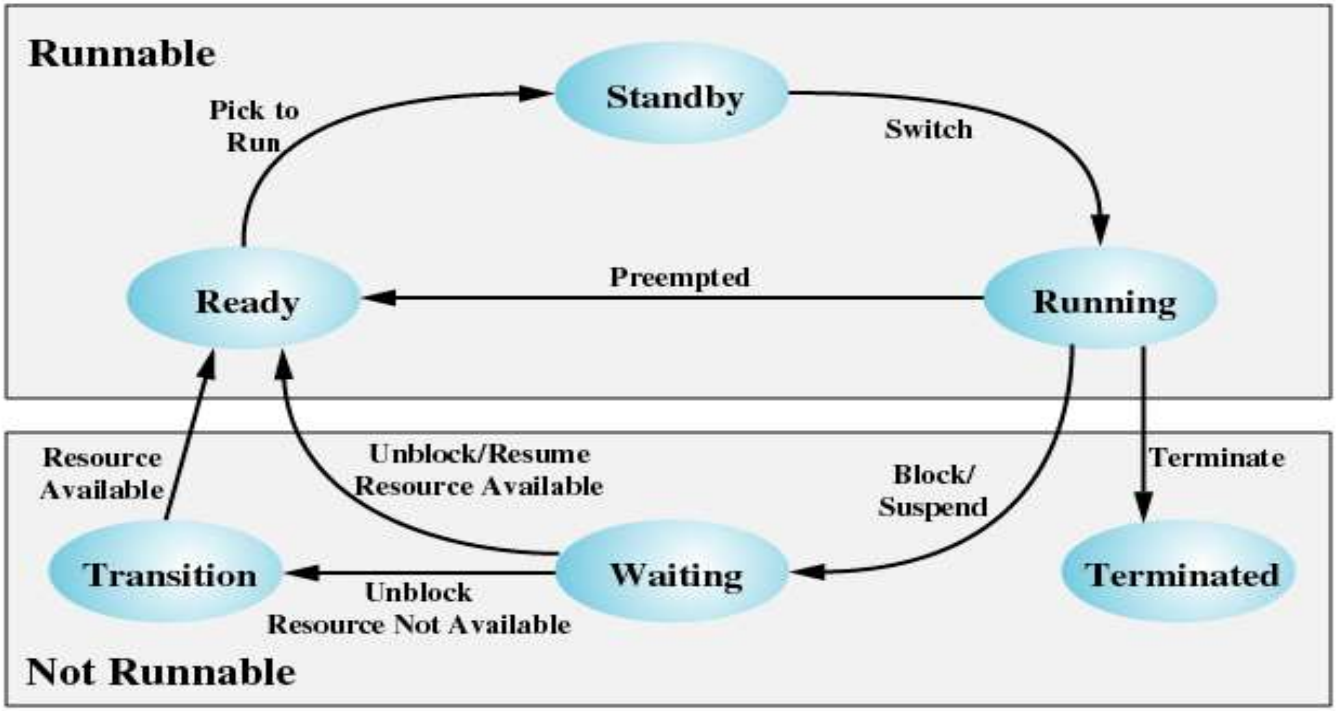
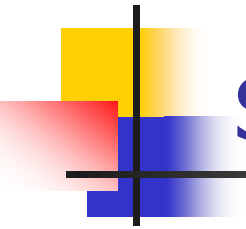


Figure 4.14 Windows Thread States



Solaris

- Process includes the user's address space, stack, and process control block
- User-level threads
- Lightweight processes (LWP)
- Kernel threads

Solaris Multithreaded Architecture

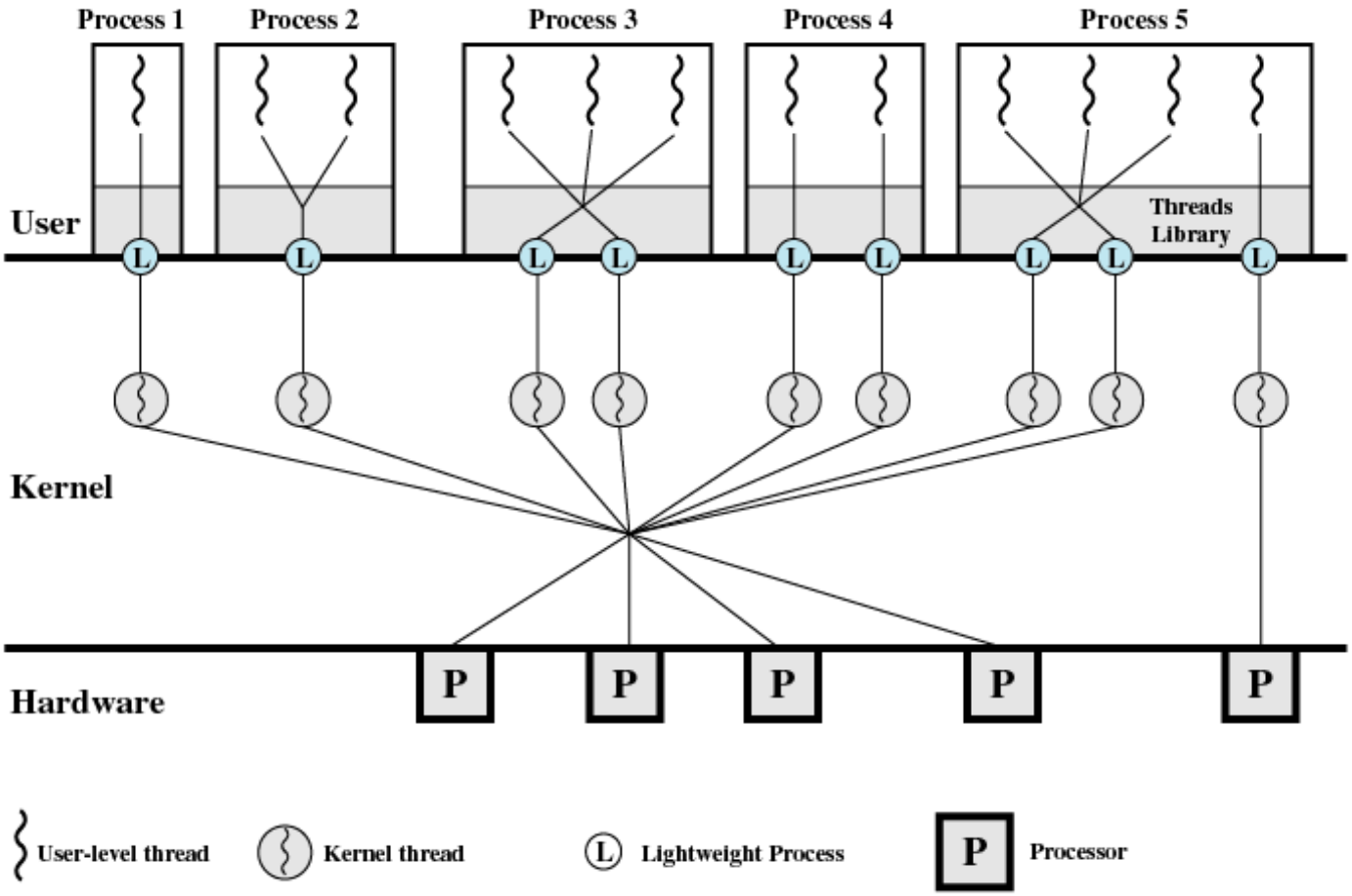


Figure 4.15 Solaris Multithreaded Architecture Example

Process Structure in Traditional Unix

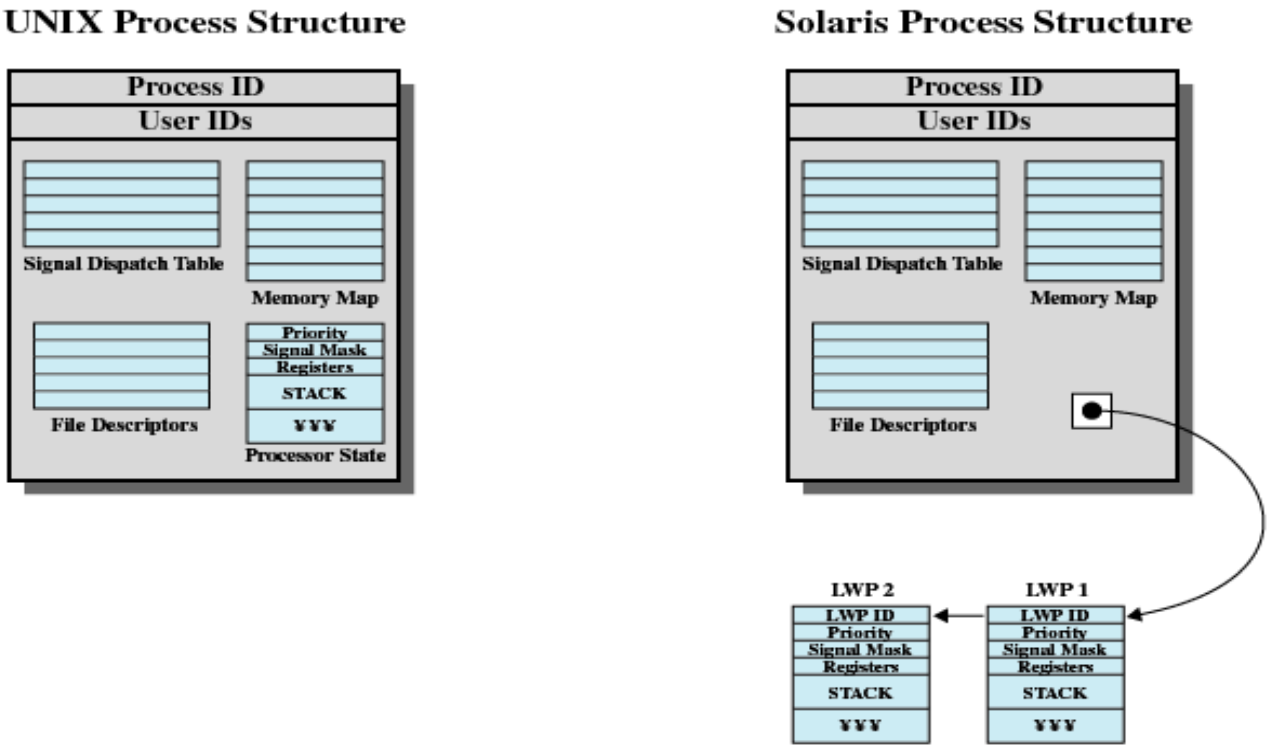


Figure 4.16 Process Structure in Traditional UNIX and Solaris [LEWI96]

Solaris Lightweight Data Structure

- Identifier
- Priority
- Signal mask
- Saved values of user-level registers
- Kernel stack
- Resource usage and profiling data
- Pointer to the corresponding kernel thread
- Pointer to the process structure

Solaris User-level Thread

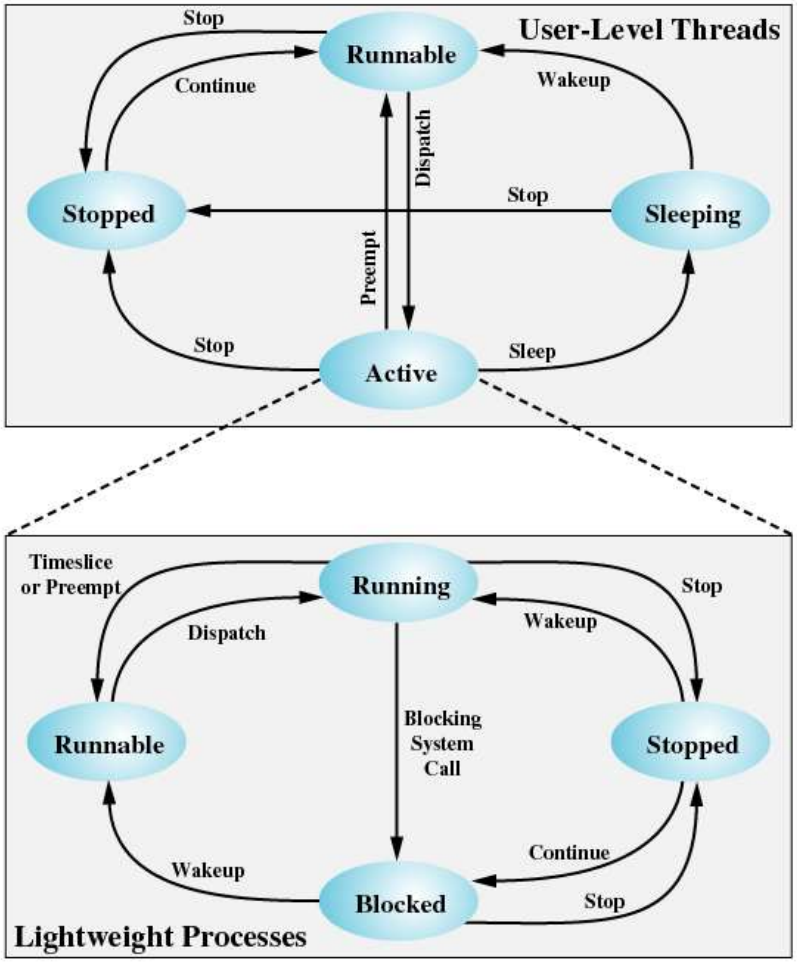
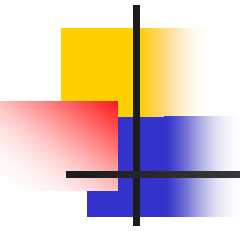
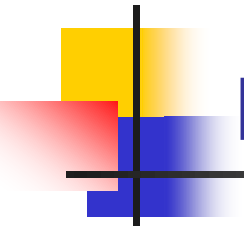


Figure 4.17 Solaris User-Level Thread and LWP States



Linux Task Data Structure

- State
- Scheduling information
- Identifiers
- Interprocess communication
- Links
- Times and timers
- File system
- Address space
- Processor-specific context



Linux States of a Process

- Running
- Interruptable
- Uninterruptable
- Stopped
- Zombie

Linux States of a Process

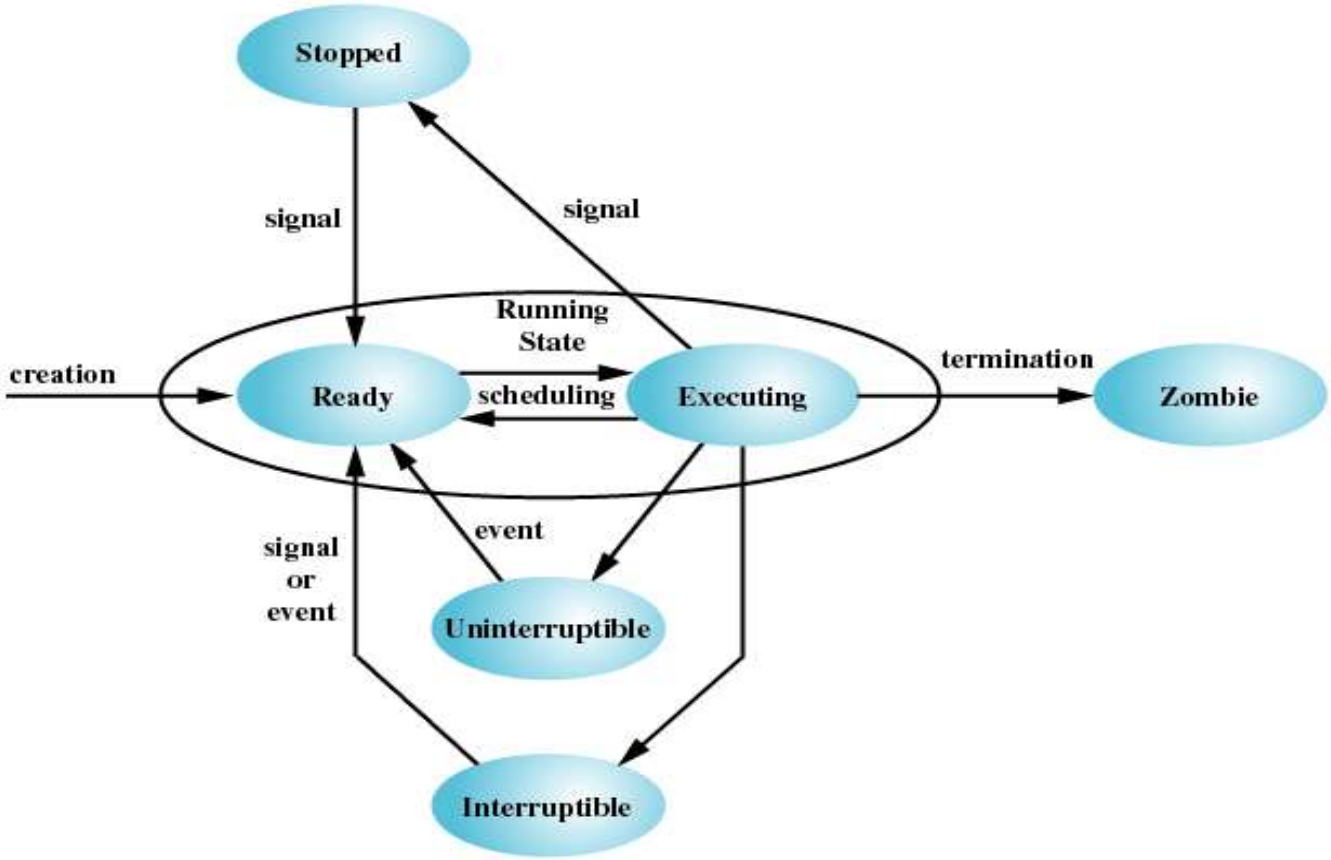


Figure 4.18 Linux Process/Thread Model